

invocare-api

Invocare l'API di dismissione (Asset-Terminator Dispatch)

Guida operativa per invocare la Function API in modo interattivo con PowerShell (`Invoke-WebRequest` / `iwr`).

1. Autenticazione

L'API è protetta **esclusivamente dalla Function Key** (`authLevel: function`). Non è configurato Easy Auth, né APIM, né autenticazione utente/OAuth.

La chiave può essere passata in due modi equivalenti:

- come query string: `?code=<functionKey>`
- come header HTTP: `x-functions-key: <functionKey>`

Negli esempi viene usata la **host key** `default`, valida per tutte le function dell'app (`WipeIntake` e `GetStatus`). Trattala come un segreto: non commitarla, non loggarla, ruotala se esposta.

Recuperare la Function Key

Dal portale Azure

1. Accedere al [portale Azure](#).
2. Aprire il Resource Group `ASSET-TERMINATOR-DISPATCH-RG`.
3. Selezionare la Function App API `attdisp-func-api-dev`.
4. Nel menu della Function App, aprire **Functions > App keys**.
5. Nella sezione **Host keys**, individuare la chiave `default` e selezionare **Show values**, quindi **Copy**.
6. Usare il valore copiato come parametro `code` oppure nell'header `x-functions-key`.

Usare la **host key** e non una chiave creata nella pagina **Functions > WipeIntake > Function Keys**: una chiave specifica di `WipeIntake` autorizza soltanto il `POST /api/v1/wipe` e non il successivo `GET /api/v1/wipe/status`.

Il nome host è visibile nella pagina **Overview** della Function App, nel campo **Default domain**. La Base URL è `https://<default-domain>`.

Con Azure CLI

```
$rg = 'ASSET-TERMINATOR-DISPATCH-RG'
$api = 'attdisp-func-api-dev'
$sub = 'b45c5b53-d8f3-4a4c-9fe5-5537818a9886'

$key = az functionapp keys list -g $rg -n $api --subscription $sub --query functionKeys.default -o tsv
$fqdn = az functionapp show -g $rg -n $api --subscription $sub --query defaultHostName -o tsv
```

2. Endpoint

Azione	Metodo	Route
Avviare una richiesta	POST	/api/v1/wipe
Consultare lo stato	GET	/api/v1/wipe/status

Base URL: `https://<fqdn>` (es. `https://attdisp-func-api-dev.azurewebsites.net`).

3. Payload della richiesta (POST /api/v1/wipe)

Campo	Obbligatorio	Note
serialNumber	uno tra	Identificativo device. In alternativa <code>imei</code> , <code>managedDeviceId</code> o <code>deviceName</code> .
operatingSystem	sì	<code>Windows</code> / <code>Apple</code> (<code>macos/ios/ipados</code>) / <code>Android</code> / <code>Mobile</code> (aliases: <code>win...</code>).
scenario	sì	<code>Retirement</code> , <code>Sale</code> , <code>Disposal</code> , <code>LostStolen</code> .
userConfirmed	consigliato	<code>true</code> per superare il guardrail di conferma utente (richiesto se <code>dryRun=false</code>).
dryRun	no	<code>true</code> = simulazione, nessun wipe reale. Default da app setting <code>DEFAULT_DRY_RUN</code> .
mdmServerId	Apple	ID server ABM/DEP, usato dai runbook Apple.
imei	no	Propagato nel messaggio canonico.
requestId	no	Se omesso viene generato un GUID. Usato per idempotenza .
callbackUrl	no	URL notificato a fine elaborazione (se configurato).

Risposta tipica: **202 Accepted** con `requestId`, `status: Queued`, l'esito dei guardrail e un header `Location` verso lo stato.

4. Invocazione interattiva con iwr

4.1 Avviare una richiesta (dry-run)

```
$body = @{
    serialNumber      = '8393-2244-5862-5920-6458-3618-53'
    operatingSystem   = 'Windows'
    scenario          = 'Disposal'
    userConfirmed     = $true
    dryRun            = $true
} | ConvertTo-Json

$resp = iwr -Uri "https://$fqdn/api/v1/wipe?code=$key" `
    -Method Post `
    -ContentType 'application/json' `
    -Body $body

$resp.StatusCode          # 202
$result = $resp.Content | ConvertFrom-Json
$result.requestId
$result.status             # Queued
```

In alternativa, con la chiave nell'header:

```
$resp = iwr -Uri "https://$fqdn/api/v1/wipe" `
    -Method Post `
    -Headers @{ 'x-functions-key' = $key } `
    -ContentType 'application/json' `
    -Body $body
```

4.2 Consultare lo stato

```
$reqId = $result.requestId
$stResp = iwr -Uri "https://$fqdn/api/v1/wipe/status?requestId=$reqId&code=$key" -Method Get
$state = $stResp.Content | ConvertFrom-Json
$state.status          # Queued | Running | Completed | Failed | Rejected
$state.automationJobName
$state.errorMessage
$state.result
```

5. Script interattivo completo (invio + polling)

```
param(
    [Parameter(Mandatory)] [string] $SerialNumber,
    [ValidateSet('Windows','Apple','Android','Mobile')] [string] $OperatingSystem = 'Windows',
    [ValidateSet('Retirement','Sale','Disposal','LostStolen')] [string] $Scenario = 'Disposal',
    [switch] $Real,                                     # senza -Real la richiesta è dry-run
    [string] $MdmServerId,
    [int] $TimeoutSeconds = 600
)

$rg = 'ASSET-TERMINATOR-DISPATCH-RG'; $api = 'attdisp-func-api-dev'
$sub = 'b45c5b53-d8f3-4a4c-9fe5-5537818a9886'

$key = az functionapp keys list -g $rg -n $api --subscription $sub --query functionKeys.default
      -o tsv
$fqdn = az functionapp show -g $rg -n $api --subscription $sub --query defaultHostName -o tsv

$payload = @{
    serialNumber      = $SerialNumber
    operatingSystem   = $OperatingSystem
    scenario          = $Scenario
    userConfirmed     = $true
    dryRun            = (-not $Real)
}

if ($MdmServerId) { $payload.mdmServerId = $MdmServerId }

Write-Host "-> POST /api/v1/wipe (dryRun=$(if (-not $Real) { '' } else { '-Real' }))" -ForegroundColor Cyan
$resp = iwr -Uri "https://$fqdn/api/v1/wipe?code=$key" -Method Post `
        -ContentType 'application/json' -Body ($payload | ConvertTo-Json)

$accepted = $resp.Content | ConvertFrom-Json
$reqId = $accepted.requestId
Write-Host "    requestId = $reqId | status = $($accepted.status)" -ForegroundColor Green

$deadline = (Get-Date).AddSeconds($TimeoutSeconds)
do {
    Start-Sleep -Seconds 10
    $st = (iwr -Uri "https://$fqdn/api/v1/wipe/status?requestId=$reqId&code=$key" -Method
        Get).Content | ConvertFrom-Json
    Write-Host ("    [{0:HH:mm:ss}] status = {1}" -f (Get-Date), $st.status)
} until ($st.status -in 'Completed','Failed','Rejected' -or (Get-Date) -gt $deadline)

Write-Host "`n=== Esito finale ===" -ForegroundColor Cyan
$st | Format-List requestId, status, platform, automationJobName, errorMessage, result
```

Uso:

```
# Simulazione (default, nessun wipe)
.\Invoke-Wipe.ps1 -SerialNumber '8393-2244-5862-5920-6458-3618-53'

# Esecuzione reale (richiede prerequisiti Automation: moduli Graph, certificato, variabili segrete)
.\Invoke-Wipe.ps1 -SerialNumber '8393-...' -Scenario Disposal -Real
```

6. Codici di risposta

Codice	Significato
202	Richiesta accettata e accodata su Service Bus.
200	<code>requestId</code> duplicato: ritorna lo stato esistente (idempotenza).
400	Payload non valido (JSON, campi obbligatori o scenario/OS non riconosciuti).
422	Respinta: device non gestito da Intune, piattaforma ambigua o guardrail KO.
502	Errore nell'interrogazione di Microsoft Graph.
500	Errore interno (persistenza stato o pubblicazione messaggio).

7. Note

- In `dryRun=true` i guardrail vengono valutati ma non bloccano l'accodamento; in `dryRun=false` un guardrail fallito produce `422 Rejected`.
- Lo stato è consultabile in qualsiasi momento tramite `requestId`.
- Per l'esecuzione **reale** dei runbook servono i prerequisiti descritti nel README (import del modulo `Microsoft.Graph.Authentication`, certificato per l'auth app-only e valorizzazione delle Automation Variables segrete).